Documentation Issues — Milestone 3

The following issues serve as a guide for each team member to choose from when contributing to the documentation branch for Milestone 3. Each issue has its corresponding section, which needs to be worked on. Make sure to add more objectives, solutions, risks/blockers, and success criteria for the selected issue on GitHub.

NOTE: Write your name in the selected issue highlighted. For example: 0. Name of issue to be worked on (Name)

1. Resolve Milestone 2 Pending Feedback (Section 1.1 - Team & Document Hygiene)

Objective

Address every reviewer comment from Milestone 2 that was deferred or marked unresolved before adding new Milestone 3 content. The document should enter M3 with a clean comment thread so new feedback isn't buried under old issues.

Proposed Solutions

- Export the list of open comments from the M2 review and assign each to a team member.
- Track each resolution as a separate commit referencing the comment ID in the message.
- Update author attribution in Section 1.1 so every contributor for M3 is listed with role and contact.

Potential Risks/Blockers

- Comments left open after M3 starts will be confused with new ones and missed during grading.
- Re-opening resolved threads if changes overwrite prior fixes.

Success Criteria

- Zero open comments inherited from M2 in the AsciiDoc source.
- Section 1.1 reflects the current M3 team roster, not the M2 one.

2. Update Derived Goals to Reflect Implementation Reality (Section 1.5 - Derived Goals)

Objective

Revisit the derived goals declared in M1/M2 and reconcile them with what the team has actually built so far. Goals that turned out to be infeasible should be removed or revised, and any goals that emerged from implementation work should be added.

Proposed Solutions

- List each derived goal and mark it Met / Partially Met / Dropped / New, with one sentence of evidence.
- For dropped goals, document the reason (scope, time, technical constraint) so the audit trail is intact.
- For new goals, link each one to the requirement or user need that motivated it.

Potential Risks/Blockers

- Silently dropping goals without justification is treated as scope drift by reviewers.
- Goals worded too broadly become impossible to validate in Section 3.2.

Success Criteria

- Every M1/M2 derived goal has an explicit status in M3.
- New derived goals are traceable back to a documented requirement.

3. Expand Domain Description with Use Scenarios (Section 2.1 - Domain Description)

Objective

Move Section 2.1 from a high-level description of the home inventory domain to a scenario-based account showing how real users interact with the inventory in concrete situations (moving, insurance claim, lending an item, monthly audit).

Proposed Solutions

- Add at least three end-to-end usage scenarios with named personas.
- For each scenario, identify the inventory operations triggered (add, locate, transfer, retire).
- Cross-reference each scenario to the corresponding entries in 2.1.2 Terminology.

Potential Risks/Blockers

- Scenarios that duplicate the requirements list add length without adding insight.
- Personas not grounded in stakeholder data (Section 4.1) will read as fabricated.

Success Criteria

- Section 2.1 contains 3+ scenarios, each tied to a persona from Section 4.1.
- Every domain term used in a scenario appears in 2.1.2.

4. Complete Terminology Glossary (Section 2.1.2 - Terminology)

Objective

Close every gap in the terminology section so any term used elsewhere in the document is defined exactly once and consistently. Reviewers in M2 flagged inconsistent use of inventory-domain terms; M3 must clean this up.

Proposed Solutions

- Scan the full document for capitalized or domain-specific terms and check each against the glossary.
- Merge synonyms (e.g., "item" vs. "asset" vs. "belonging") and pick one canonical term.
- Add a short example sentence to each glossary entry to disambiguate edge cases.

Potential Risks/Blockers

- Renaming a canonical term without sweeping the document creates new inconsistencies.
- Overly academic definitions that don't match how the term is used in the requirements.

Success Criteria

- No undefined domain term appears in any other section.
- Each glossary entry has a definition AND a usage example.

5. Strengthen Functional Requirements Coverage (Section 2.2 - Requirements)

Objective

Audit the functional requirements list for completeness against the Milestone 3 feature set. Every feature being implemented in this milestone must trace back to a numbered requirement; every requirement must have an owner and a priority.

Proposed Solutions

- Produce a requirements coverage table: requirement ID → feature → owner → priority → milestone.
- Flag requirements that are too vague to test and rewrite them in SMART form.
- Add any requirements discovered during M2 implementation that were never formally documented.

Potential Risks/Blockers

- Requirements written after the code is built tend to describe the code, not the user need.
- Missing priorities make it impossible to defend scope cuts later.

Success Criteria

- 100% of M3 features map to a numbered requirement.
- Every requirement has owner, priority, and a testable acceptance condition.

6. Document Machine and Deployment Requirements (Section 2.2.5 - Machine Requirements)

Objective

Specify the minimum and recommended environment for running the home inventory system — both for end users (device, OS, network) and for the team's deployment target. M2 left this section as a stub.

Proposed Solutions

- List supported platforms with versions (mobile OS, desktop browser, server runtime).
- Document storage and bandwidth assumptions for the inventory data set sizes the team is targeting.
- Note any third-party services the deployment depends on and their pricing tier.

Potential Risks/Blockers

- Listing requirements the team hasn't actually tested on (e.g., browsers nobody runs).
- Omitting offline / low-connectivity behavior, which is realistic for home inventory use.

Success Criteria

- Section 2.2.5 specifies versions, not just product names.
- Each platform claim is supported by an actual test run noted in 3.2.

7. Add State Diagrams for the Item Lifecycle (Section 2.3 - Implementation)

Objective

Produce state diagrams covering the lifecycle of an inventory item from acquisition through disposal, including the lent, lost, and archived states. M2 covered the data model but not the state behavior.

Proposed Solutions

- Identify every state an item can be in and the events that trigger transitions.
- Draw the diagram in a tool that exports to a format the AsciiDoc build can render (PlantUML, Mermaid).
- Reference the state diagram from the corresponding requirements in Section 2.2.

Potential Risks/Blockers

- Diagrams that show states without showing the triggering events are not useful for implementation.
- Hand-drawn PNGs that drift out of sync with the documented states.

Success Criteria

- At least one state diagram for the item lifecycle, rendered from source in the repo.
- Every state and transition is referenced in either Section 2.2 or 2.3.

8. Add Sequence Diagrams for Core Workflows (Section 2.3 - Implementation)

Objective

Document the principal end-to-end workflows (add item, search item, generate report) as sequence diagrams showing the user, the application, and any backend or external service involved.

Proposed Solutions

- Pick the three highest-priority workflows from the requirements in Section 2.2.
- For each, draw a sequence diagram with explicit actors, messages, and return values.
- Annotate each diagram with the requirement ID it implements.

Potential Risks/Blockers

- Sequence diagrams that stop at the UI layer don't show enough to validate the implementation.
- Diagrams produced for a feature that has since been redesigned.

Success Criteria

- 3 sequence diagrams covering the prioritized workflows.
- Each diagram is linked from the relevant requirement and from Section 3.2.

9. Document the Data Model and Persistence Layer (Section 2.3 - Implementation)

Objective

Add a definitive description of the data model — entities, attributes, relationships, and persistence choices — that reflects what is actually in the codebase as of M3. Replace any placeholder ER diagram from M2.

Proposed Solutions

- Generate the schema from the current implementation rather than transcribing it by hand.
- Include cardinality and ownership for every relationship.
- Note migration strategy for any field renamed or removed since M2.

Potential Risks/Blockers

- A data model documented from memory will diverge from the running schema by M4.
- Skipping migration notes makes it impossible to grade backward compatibility claims.

Success Criteria

- ER diagram is auto-generated or has a script in the repo that regenerates it.
- Every entity in the diagram maps to a class or table in the implementation.

10. Add Architecture Overview Diagram (Section 2.3 - Implementation)

Objective

Provide a single-page architecture diagram showing how the major components of the system fit together: client, server, persistence, external services, and any background jobs.

Proposed Solutions

- Use the C4 model's container view as a starting template.
- Label each component with the technology it uses and the protocol on each connection.
- Mark which components exist today vs. which are planned for M4.

Potential Risks/Blockers

- Marketing-style diagrams with logos and clouds but no protocol detail are not useful.
- An aspirational architecture that doesn't match what's deployed will mislead reviewers.

Success Criteria

- One architecture diagram showing today's deployment, with all components labeled.
- Planned-but-not-built components are clearly marked as such.

11. Document Implementation Decisions and Trade-offs (Section 2.3 - Implementation)

Objective

Capture the non-obvious technical decisions made during M2 and M3 implementation — framework choices, library swaps, data format decisions — together with the alternatives that were rejected and why.

Proposed Solutions

- Write one short ADR (Architecture Decision Record) per significant decision.
- Reference the requirements or constraints that drove the decision.
- Note the date and the team members involved in each decision.

Potential Risks/Blockers

- ADRs written long after the decision lose the original reasoning.
- Treating every minor choice as worthy of an ADR dilutes the section.

Success Criteria

- At least one ADR per major subsystem (frontend, backend, storage, auth).

- Each ADR identifies the alternatives considered, not only the chosen path.

12. Expand Concept Analysis with Implementation Evidence (Section 3.1 - Concept Analysis)

Objective

Tighten Section 3.1 so each domain concept is connected to concrete code or documentation in the M3 deliverable. Concepts that have not been implemented should be moved to a backlog or removed.

Proposed Solutions

- For every concept, link to the class, module, or document that realizes it.
- Add a short paragraph noting any concept whose meaning changed since M2.
- Remove orphan concepts that have no realization and aren't planned for M4.

Potential Risks/Blockers

- Linking to a file path that moves later will create broken references; prefer stable identifiers.
- Hiding unimplemented concepts instead of deferring them masks scope problems.

Success Criteria

- Every concept in 3.1 has either an implementation link or a deferred status.
- No orphan concepts remain in the section.

The issues below carry a heavier weight.

13. Establish a Documentation Inspection Process (Section 3.2 - Validation & Verification)

Objective

Define and run a formal inspection process for the documentation itself before M3 submission. Inspections catch defects (ambiguity, contradiction, missing rationale) that grading reviewers would otherwise find.

Proposed Solutions

- Adopt a checklist-based inspection covering clarity, traceability, consistency, and completeness.
- Pair each section with a designated inspector who did not author it.
- Log every defect found and the commit that resolved it in an inspection table inside 3.2.

Potential Risks/Blockers

- Inspections done by the author against their own section catch nothing.
- A defect log that records findings but not resolutions is incomplete evidence.

Success Criteria

- Inspection table in 3.2 with one row per defect: section, finding, inspector, resolution commit.
- Every section of the document has been inspected by someone other than its author.

14. Document Validation Activities with Evidence (Section 3.2 - Validation & Verification)

Objective

Separate validation (are we building the right thing?) from verification (are we building it right?) and provide concrete evidence for each validation activity completed during M3 — stakeholder feedback, scenario walkthroughs, usability checks.

Proposed Solutions

- List each validation activity with date, participants, and outcome.
- Attach or link the artifact produced (feedback notes, recorded walkthrough, survey results).
- Map each activity to the requirements or scenarios it validated.

Potential Risks/Blockers

- Describing validation as a future plan rather than completed work — M2 reviewers flagged this.
- Stakeholder feedback collected informally with no record is not defensible evidence.

Success Criteria

- Section 3.2 contains a validation log with at least three completed activities.
- Each activity links to a stored artifact and the requirements it covers.

15. Document Verification Activities with Test Coverage (Section 3.2 - Validation & Verification)

Objective

Specify the verification strategy for the implementation — unit, integration, and acceptance — and present coverage data for the M3 deliverable. The goal is reviewer-grade evidence that the code does what the requirements say.

Proposed Solutions

- Adopt a test-categorization scheme and tag each test in the codebase accordingly.
- Report coverage per requirement, not just per file.
- Flag requirements with no automated verification and note the manual procedure used instead.

Potential Risks/Blockers

- Aggregate line coverage hides requirements with zero test exercise.
- Manual test scripts that aren't checked into the repo can't be re-run by reviewers.

Success Criteria

- A coverage report per requirement is included or linked from Section 3.2.
- Every requirement is either covered by automated tests or has a checked-in manual procedure.

16. Reconcile Stakeholder Identification with Current Reality (Section 4.1 - Stakeholder Identification)

Objective

Refresh the stakeholder list to reflect who has actually engaged with the project through M2 and M3. Add any new stakeholders surfaced during requirements acquisition, and adjust influence/interest classifications based on observed behavior.

Proposed Solutions

- Review each stakeholder entry against the actual feedback collected in 3.2 and 4.4.
- Add new stakeholders introduced through implementation work (hosting providers, library maintainers if relevant).
- Document the engagement channel for each stakeholder (email, interview, survey, observation).

Potential Risks/Blockers

- Stakeholder lists drift toward aspirational; reviewers expect lists grounded in actual contact.
- Demoting a stakeholder without documenting why looks like cherry-picking.

Success Criteria

- Every stakeholder in 4.1 has a documented engagement channel.
- Classification changes since M2 have a one-sentence justification.

17. Close the Requirements Acquisition Loop (Section 4.4 - Domain Requirements Acquisition)

Objective

Run a second round of requirements acquisition focused on the gaps and ambiguities discovered during M2 implementation, and document both the questions asked and the answers received.

Proposed Solutions

- List the open questions inherited from M2 with their priority.
- Conduct interviews, surveys, or observation sessions targeting those questions.
- Record outcomes and update Section 2.2 with any new or revised requirements.

Potential Risks/Blockers

- Asking the same questions as M1 yields the same answers; questions must reflect what implementation revealed.
- Updates to 2.2 without updating 3.1 or 3.2 create traceability gaps.

Success Criteria

- Section 4.4 contains a Q&A table from the M3 acquisition round.
- Every requirement added or revised in 2.2 is cited in 4.4.

18. Build a Requirements Traceability Matrix (Section 2.2 - Requirements)

Objective

Produce a single matrix that maps each requirement to its source stakeholder (Section 4.1), its validation activity (Section 3.2), its verification artifact (Section 3.2), and its implementation locus (Section 2.3). The matrix is the integration point for the document.

Proposed Solutions

- Store the matrix as a structured file (CSV or AsciiDoc table) so it can be regenerated.
- Use stable identifiers for requirements, not section numbers.
- Add a column flagging any requirement that fails traceability in any dimension.

Potential Risks/Blockers

- Maintaining the matrix by hand in two places guarantees they will diverge.
- Requirements without a stakeholder source are an indication that 4.1 or 4.4 is incomplete.

Success Criteria

- One traceability matrix in the repo, rendered into 2.2.
- Every row has a value in every column or an explicit gap reason.

19. Update the Terminology Section with M2/M3 Vocabulary (Section 2.1.2 - Terminology)

Objective

Capture every term introduced or refined during M2 and M3 — including implementation-specific terms (e.g., service names, integration patterns) — and resolve any conflicts with earlier definitions.

Proposed Solutions

- Diff the M2 terminology against terms used in current document drafts.
- Mark each term with its source section so reviewers can audit.
- Resolve synonyms: pick one canonical term and flag the alternatives.

Potential Risks/Blockers

- A bloated glossary with marketing synonyms is worse than a small precise one.
- Terms that are only used in one place don't need to be in the glossary.

Success Criteria

- Every term used in 2 or more sections appears in 2.1.2 with a single canonical definition.
- No synonyms remain in the running text without justification.

20. Document the Build and Deployment Procedure (Section 2.3 - Implementation)

Objective

Write a reproducible build and deployment procedure so any team member or grader can stand up a working instance of the M3 deliverable from a clean machine.

Proposed Solutions

- Capture prerequisites (OS, runtime versions, accounts needed).
- Provide commands in copy-paste form, ordered and tested on a clean environment.
- Note known failure modes and recovery steps.

Potential Risks/Blockers

- Steps that work on the author's machine but rely on uncommitted local config will fail elsewhere.
- Out-of-date version numbers in prerequisites cause silent breakage.

Success Criteria

- A second team member follows the procedure end-to-end on a clean environment and succeeds.
- The procedure is committed to the repo alongside the documentation.

21. Produce an Other Activities Log for M3 (Section 1.4 - Other Activities)

Objective

Maintain an honest log of non-coding effort during M3 — research, meetings, training, blocked time — so reviewers can assess team capacity and effort distribution.

Proposed Solutions

- Use a single shared log with date, member, activity, and rough hours.
- Aggregate into a per-member summary at the end of the milestone.
- Note any blocker that consumed more than half a day of any member's time.

Potential Risks/Blockers

- A retrospective log written the night before submission will be both incomplete and inaccurate.
- Padding the log with trivial entries reduces its credibility.

Success Criteria

- Section 1.4 contains an activity table with per-member totals.
- Each significant blocker has its own entry referencing the issue it impacted.

22. Integrate, Proofread, and Tag the M3 Document (Section 1.1 - Team)

Objective

Run a final integration pass over the M3 document — consistent voice, consistent figure numbering, no broken cross-references — and tag the repository so the submitted version is recoverable.

Proposed Solutions

- Designate an integration editor distinct from individual section authors.
- Run the AsciiDoc build, fix every warning, and resolve every TODO comment.
- Create a git tag (e.g., m3-final) on the exact commit submitted.

Potential Risks/Blockers

- Submitting from HEAD without a tag makes it impossible to identify the graded version after later commits.
- Multiple voices in one section read as careless even when content is correct.

Success Criteria

- AsciiDoc build produces zero warnings on the tagged commit.
- The repository has a git tag matching the submitted PDF.